

Java

Collections

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.LinkedList;
import java.util.TreeSet;

public class Collection {

    public static void main(String[] args) {
        System.out.println("Collections");

        // array => array is used to store multiple values of same data type

        // arraylist => dynamic size
        // util package
        ArrayList<Integer> user = new ArrayList<>();
        LinkedList<String> ll = new LinkedList<>();
        HashSet<String> hs = new HashSet<>(); // insertion order not maintained
        LinkedHashSet<String> lhs = new LinkedHashSet<>(); // maintain insertion
order
        TreeSet<Integer> ths = new TreeSet<>(); // automatically sorts elements

        ths.add(100);
        ths.add(3);
        ths.add(4);
        // ths.add(null); // throws null pointer exception

        System.out.println(ths.headSet(5)); // less than the given number
        System.out.println(ths.tailSet(5)); // greater than the given number
        System.out.println(ths.subSet(4, 10)); // ranges

        System.out.println(ths);

        lhs.add("Ravi");
        lhs.add("Ravi");
        lhs.add(null);

        System.out.println(lhs);

        hs.add("Ravi");
        hs.add("Ravi");
        hs.add(null);

        System.out.println(hs);
```

```
ll.add("FIIT");
ll.add("Academy");
ll.add("java");

ll.addFirst("first");
ll.addLast("last");

// ll.remove(4);

System.out.println(ll.peek());

// System.out.println(ll.poll());
System.out.println(ll.pop());

System.out.println(ll);

// list interface => it maintains insertion order
// it allow duplicates values

user.add(1);
user.add(4);
user.add(3);
user.add(2);
user.add(4);

user.set(2, 10);

System.out.println(user.contains(100));
System.out.println(user);

user.remove(0);

System.out.println(user.get(0));
System.out.println(user.get(2));
user.clear();
System.out.println(user.size());

// if you need to store values in dynamic
// random access of elements using index
}

}
```